



Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
Information Technology department
ACADEMIC YEAR 2017-2018

Class –T.E. Semester –V Subject: Operating Systems

Course Code: ITC502 Synoptic answer MSE

Synoptic answer MSE.

Operating Systems TEIT Vth semIT

1. (a) In a multiprogramming and time sharing environment, several users share the system simultaneously. What are the two security problems that result in this situation? (1)-CO1

Answer:

1. One user can read the private data of another user - privacy.
2. One user can corrupt the private data of another user - integrity.
3. One user can prevent another user from getting anything done - denial of service.

(b) mention any 2 main objectives of OS? (1)

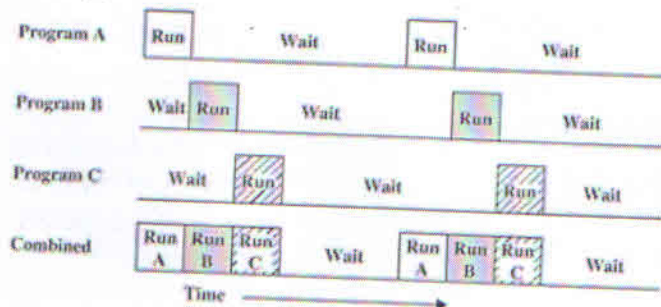
Answer:

1. Ability to evolve
2. Convenient to use.
3. Efficient

(c) Explain the need of multiprogramming in OS with example? (2)

Answer: 1. Keep the CPU busy as long as there are processes ready to execute - increase CPU utilization (1) mark

Diagram (1) mark



(d) What does fork() do? Hence, what will be the output of the following program? (2)

Answer: fork() creates child process. (1/2) mark

o/p (1/2) mark

o/p: Hello world!

Hello world!

Parent process prints Hello world! and the child process also prints Hello world!

1 (mark) explanation

2. a.) gantt chart (1)mark
calculation (1)mark

Let three processes be p0, p1 and p2. Their execution time is 10, 20 and 30 respectively. p0 spends first 2 time units in I/O, 7 units of CPU time and finally 1 unit in I/O. p1 spends first 4 units in I/O, 14 units of CPU time and finally 2 units in I/O. p2 spends first 6 units in I/O, 21 units of CPU time and finally 3 units in I/O.

idle p0 p1 p2 idle
0 2 9 23 44 47
Total time spent = 47
Busy time = $(42/47) * 100 = 89.36$

Each one scheduler(1) mark

LTS: long term scheduler: very low frequency: to select process to be loaded in MM.

STS: short term scheduler: very high frequency: to select next process for scheduling.

MTS: medium term scheduler: medium frequency: select process to swap from MM to secondary storage.

OR

I. If a process makes a transition D, it would result in another process making transition A immediately. - This is false. It is not said anywhere that one process terminates, another process immediately come into Ready state. It depends on availability of process to run & Long term Scheduler.

II. A process P2 in blocked state can make transition E while another process P2 is in running state. - This is correct. There is no dependency between running process & Process getting out of blocked state.

III. The OS uses preemptive scheduling. :- This is true because we got transition C from Running to Ready.

IV: The OS uses non-preemptive scheduling. Well as previous statement is true, this becomes false.

So answer is C) II and III .

Each option explanation 1 mark and final correct option 1 mark

b) Differentiate between User Level Thread and Kernel Level Thread .(4)

ULT	KLT
user level threads are managed by a user level library however, they still require a kernel system call to operate. They are faster. Creating threads, switching between threads and synchronizing threads only needs a procedure call.	They are slower than user level threads due to the management overhead. Kernel level context switch involves more steps than just saving some registers. Finally, they are not portable because the implementation is operating system dependent.
Scheduling can be application specific. The scheduling algorithm can be tailored to the application without disturbing the underlying OS scheduler	Kernel level threads are managed by the OS, therefore, thread operations (ex. Scheduling) are implemented in the kernel code.
ULTs can run on any OS. No changes are required to the underlying kernel to support ULTs. The threads library is	They are not portable because the implementation is operating system dependent. KLT can't run on any OS.

a set of application-level functions shared by all applications.	
In a typical OS, many system calls are blocking. As a result, when a ULT executes a system call, not only is that thread blocked, but also all of the threads within the process are blocked	Kernel can schedule another thread if one thread of a process is block.

c) Write down two advantage of Shortest Job First . (1) mark

$\alpha = 0.5$, $\tau_1 = 10$, actual burst times $(t_1, t_2, t_3, t_4) = (4, 8, 6, 7)$. Calculate τ_5 . (2) mark
 Advantage: Maximum throughput, Minimum- wait time, turn around time.

$$\tau_2 = 0.5 * 4 + 0.5 * 10 = 7$$

$$\tau_3 = 0.5 * 8 + 0.5 * 7 = 7.5$$

$$\tau_4 = 0.5 * 6 + 0.5 * 7.5 = 6.75$$

$$\tau_5 = 0.5 * 7 + 0.5 * 6.75 = 6.875$$

Q.3. a) .

Sr. no. 1,3,4, leads to deadlock.

In 1st case Process X,Y,Z acquire 1 resource and wait for other process to release the required resource. So no one can progress further. So it leads to deadlock.

In 2nd case there will be tie between X and Y. Any one may win. Suppose X wins. So y will be out of race. Now suppose Z perform p(a). Now X can't perform p(a) again. So it will be out of race. So Z will complete and rest all can complete the execution. So no deadlock.

In 3rd case Each process acquire a resource and wait for others to release so leading to deadlock.

In 4th case There is tie between Y and Z for p(c). Suppose Y wins and Z will be out of race. Now if X runs and it performs p(a) and p(b). Now X will wait for Y to release c and Y will wait for X to release b. so it leads to deadlock.

Each option justification 1 mark. Choosing correct option 1 mark

OR

Above construct ensure Mutual exclusion but could not prevent deadlock. (1) mark

P1 can do wants1 = true and then P2 can do wants2 = true. Now, both P1 and P2 will be waiting in the while loop indefinitely without any progress of the system - deadlock. (1) mark

When P1 is entering critical section it is guaranteed that wants1 = true (wants2 can be either true or false). So, this ensures P2 won't be entering the critical section at the same time. In the same way, when P2 is in critical section, P1 won't be able to enter critical section. So, mutual exclusion condition satisfied. (1) mark

Part of the code where shared data is stored is called critical section. (1/2) mark

Several processes access and manipulate the same data concurrently and outcome of execution depends on the order in which access takes place is called race condition. (1/2) mark

Bounded wait: Process waiting to enter CS will get chance to execute within specified time period. (1/2) mark

Eg. There are n process p1, p2, ..., pn. So all process waiting to enter CS will definitely get chance to enter CS within specific time period. (1/2) mark

Progress: who will enter next CS will be decided by processes who want to enter CS and not all processes. (1/2) mark

Eg. There are n process p1, p2, ..., pn but only 3 process want to enter CS. Then only those 3 process will decide who will enter CS and not all n process. (1/2) mark.

b)

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

m=3, n=5 Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For i=0 Step 2

Need₀ = 7, 4, 3

7, 4, 3 3, 3, 2

Finish [0] is false and Need₀ > Work

So P₀ must wait But Need ≤ Work

For i=1 Step 2

Need₁ = 1, 2, 2

1, 2, 2 3, 3, 2

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

3, 3, 2 2, 0, 0 Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For i=2 Step 2

Need₂ = 6, 0, 0

6, 0, 0 5, 3, 2

Finish [2] is false and Need₂ > Work

So P₂ must wait

For i=3 Step 2

Need₃ = 0, 1, 1

0, 1, 1 5, 3, 2

Finish [3] is false and Need₃ < Work

So P₃ must be kept in safe sequence

5, 3, 2 2, 1, 1 Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For i=4 Step 2

Need₄ = 4, 3, 1

4, 3, 1 7, 4, 3

Finish [4] is false and Need₄ < Work

So P₄ must be kept in safe sequence

7, 4, 3 0, 0, 2 Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For i=0 Step 2

Need₀ = 7, 4, 3

7, 4, 3 7, 4, 5

Finish [0] is false and Need < Work

So P₀ must be kept in safe sequence

7, 4, 5 0, 1, 0 Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For i=2 Step 2

Need₂ = 6, 0, 0

6, 0, 0 7, 5, 5

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

7, 5, 5 3, 0, 2 Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Finish [i] = true for 0 ≤ i ≤ n Step 4

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Find need 1 mark.

System is in in safe state or not . 1 mark.

Finding safe sequence 3 mark.

c)What are the necessary conditions for deadlock?[2]

Each condition ½ marks.

1.mutual exclusion. The resources involved must be unshareable; otherwise, the processes would not be prevented from using the resource when necessary.

2. hold and wait or partial allocation. ...

3. no pre-emption. ...

4 circular wait.